Hormiguero Vivo

Simulación híbrida ABM + dinámica de sistemas de una colonia de hormigas ante sequías, inundaciones y depredadores

[Integrantes del equipo]

Modelación y Simulación
Universidad del Valle de Guatemala

[Fecha de entrega]

Resumen

Hormiguero Vivo es una simulación híbrida e interactiva de una colonia de hormigas. Cada hormiga es un agente (ABM) que explora, recoge comida, deja feromonas y huye de depredadores; el nido es un sistema de stocks y flujos (SD) de comida, huevos, larvas y obreras. Los dos modelos están acoplados: la comida que entregan los agentes alimenta el nacimiento de nuevas hormigas, y el tamaño de la colonia y su hambre cambian la fracción de hormigas que forrajea. El modelo se evalúa con tres perturbaciones (sequía, inundación e invasión de depredadores) y se mide con acumuladores exactos de población, muertes por causa y tiempo de recuperación. Con cinco semillas y 20 000 ticks por corrida, la colonia sobrevive en los cuatro escenarios; la población se estabiliza alrededor de 244 hormigas, en coincidencia con el equilibrio analítico de 250; la sequía produce una caída media de 34 % y la invasión de depredadores explica el 25 % de las muertes. El sistema incluye un backend en Python, una interfaz web en tiempo real, 136 pruebas automáticas del backend y 66 del frontend. El documento describe el problema, el modelo, las herramientas, el uso crítico de IA generativa, los resultados y sus límites.

Índice

1. Descripción del problema **3**

1.1. Relevancia 3

1.2. Alcance 3

2. Análisis **4**

2.1. Por qué un modelo híbrido 4

2.2. Variables y supuestos 4

2.3. Preguntas que el modelo debe responder 5

3. Propuesta de solución **5**

4. Descripción de la solución **5**

4.1. Arquitectura 5

4.2. El mundo 6

4.3. Las hormigas (ABM) 6

4.4. Los depredadores 7
4.5. El nido (SD) 7
4.6. Acoplamiento entre SD y ABM 8
4.7. Perturbaciones y métrica de resiliencia 9
4.8. Interfaz y contrato 9
4.9. Determinismo y rendimiento 9
4.10. Parámetros de los escenarios 9

5. Herramientas aplicadas **10**
5.1. Herramientas vistas en clase 10
5.2. Herramientas que no se vieron en clase 10
5.3. Uso de IA generativa 11

6. Resultados **13**
6.1. Método 13
6.2. Supervivencia por escenario 13
6.3. Causas de muerte 14
6.4. Verificación contra equilibrios analíticos 14
6.5. El proceso iterativo: qué cambió con las correcciones 15
6.6. Calidad técnica 16
6.7. Criterios de calibración 16

7. Conclusión **16**
7.1. Limitaciones 17
7.2. Trabajo futuro 17

1. Descripción del problema

Una colonia de hormigas es un ejemplo clásico de sistema complejo: ninguna hormiga conoce el plan de la colonia, y aun así el conjunto encuentra comida, construye rastros eficientes y se recupera de daños. El comportamiento colectivo surge de reglas locales sencillas (moverse, oler un rastro, recoger, regresar) y de una retroalimentación química: las hormigas que encuentran comida marcan el camino, y ese camino atrae a otras.

El problema que abordamos es el siguiente:

> *¿Cómo influyen el tamaño de la colonia, su estrategia de forrajeo y la comida disponible en la supervivencia de una colonia sometida a perturbaciones del entorno?*

La pregunta tiene dos escalas que se afectan mutuamente. En la escala individual, cada hormiga decide dónde moverse, si recoge comida y si huye. En la escala de la colonia, la comida acumulada determina cuántos huevos se ponen, cuántas larvas maduran y cuántas hormigas nuevas salen a forrajear. Un modelo que solo representara a los individuos no capturaría bien las acumulaciones y los retrasos del nido; un modelo que solo representara el nido en agregado no capturaría la geometría del forrajeo ni el efecto de los depredadores.

1.1. Relevancia

El tema tiene aplicación en la academia y en la industria:

- **Academia.** El forrajeo de hormigas es un problema estándar de la biología computacional y de la ciencia de sistemas complejos. Los modelos de referencia que revisamos estudian el forrajeo con feromonas [7], la energía y las colonias rivales [1], la producción de hormigas a partir de la energía del nido [2], la robustez ante feromonas atractivas y repelentes [6], el reclutamiento según el tamaño de la colonia [5], el forrajeo según la distribución de la comida [3] y la resiliencia ante depredadores [4].
- **Industria.** El comportamiento de las hormigas inspira algoritmos de optimización de rutas y de asignación de tareas, y el mismo enfoque híbrido (agentes más stocks y flujos) se usa para estudiar cadenas de suministro y poblaciones con inventarios. Una simulación interactiva también sirve como herramienta de enseñanza.

1.2. Alcance

Modelamos una sola colonia en un mundo de 200×200 celdas con un nido circular, fuentes de comida, obstáculos y depredadores. Las perturbaciones son tres: sequía, inundación y oleada de depredadores. Quedan fuera las colonias rivales, la construcción del nido, los distintos tipos de comida y la validación contra datos de campo; los discutimos como trabajo futuro (sección 7).

2. Análisis

2.1. Por qué un modelo híbrido

Tabla 1: Qué aporta cada técnica y qué deja fuera.

	Modelo basado en agentes (ABM)	**Dinámica de sistemas (SD)**
Captura	Heterogeneidad, espacio, interacción local, aleatoriedad individual	Acumulaciones, retrasos, retroalimentación agregada
Se ajusta a	Hormigas, depredadores, rastros de feromona	Reservas de comida, huevos, larvas
Limitación	Costoso de calibrar y de analizar en forma cerrada	No distingue individuos ni espacio

El forrajeo es espacial y local, por lo que se modela con agentes. La vida del nido es una cadena de acumulaciones con retrasos (un huevo tarda 200 ticks en eclosionar y una larva 400 en madurar), por lo que se modela con stocks y flujos. El reto está en el acoplamiento: el modelo SD es continuo y el ABM es discreto. Las obreras no pueden ser un stock que se integra, porque cada una es un agente con posición y energía; por eso el modelo conserva dos invariantes (sección 4.6).

2.2. Variables y supuestos

Tabla 2: Variables principales del modelo.

Tipo	**Variable**	**Descripción**
Entrada	Perturbación	Sequía, inundación u oleada de depredadores, con intensidad, duración y región
Entrada	Parámetros de la colonia	Velocidad, consumo, ritmo de puesta, esperanza de vida, fracción de soldados
Estado (SD)	F, E, L	Comida, huevos y larvas del nido
Estado (ABM)	Hormigas vivas W	Posición, rumbo, rol, estado, carga, energía y edad de cada agente
Estado (mundo)	Capas	Comida, obstáculos, agua, humedad y tres feromonas
Salida	Supervivencia	Población, pico y mínimo, muertes por causa, eficiencia de forrajeo, tiempo de recuperación

Supuestos principales:

1. El tiempo es discreto. Cada tick avanza el mundo, los agentes y el nido, y el nido se integra con el método de Euler y paso $dt = 1$.
2. Los agentes no se comunican de forma directa: toda coordinación pasa por las capas de feromona del mundo.
3. La reina no es un agente. Su puesta es un flujo del nido que depende de la comida almacenada.
4. Las hormigas nuevas aparecen dentro del nido y nacen soldado con una probabilidad fija.

5. Con poca humedad, las hormigas gastan más energía fuera del nido.

2.3. Preguntas que el modelo debe responder

1. ¿Se estabiliza la población de una colonia sin perturbaciones, y en qué valor?
2. ¿Cuánto cae la población en una sequía y cuánto tarda en recuperarse?
3. ¿Cuántas hormigas mueren por una inundación, y sobrevive la colonia?
4. ¿Qué fracción de las muertes causan los depredadores en una invasión?
5. ¿Se cumple el acoplamiento: cada larva que madura se convierte en exactamente una hormiga real?

3. Propuesta de solución

Proponemos *Hormiguero Vivo*, un programa en dos partes:

- **Motor de simulación (backend).** Un modelo híbrido ABM + SD escrito en Python con NumPy, expuesto por una API REST y un canal WebSocket.
- **Interfaz interactiva (frontend).** Una aplicación web que dibuja el mundo en tiempo real, muestra el nido como un diagrama de stocks y flujos, grafica las series en vivo y permite dibujar obstáculos, colocar comida o depredadores y programar perturbaciones.

El desarrollo siguió un enfoque *contract-first*: primero se definió el contrato de la API (esquemas y OpenAPI) con un motor simulado, y el frontend se construyó contra ese contrato mientras se implementaba el modelo real. El modelo real se dividió en tres paquetes de dificultad similar que se desarrollaron en paralelo: (1) mundo y entorno, (2) agentes ABM y (3) nido SD y acoplamiento. Esta separación obliga a que las interfaces entre paquetes sean explícitas y probadas.

4. Descripción de la solución

4.1. Arquitectura

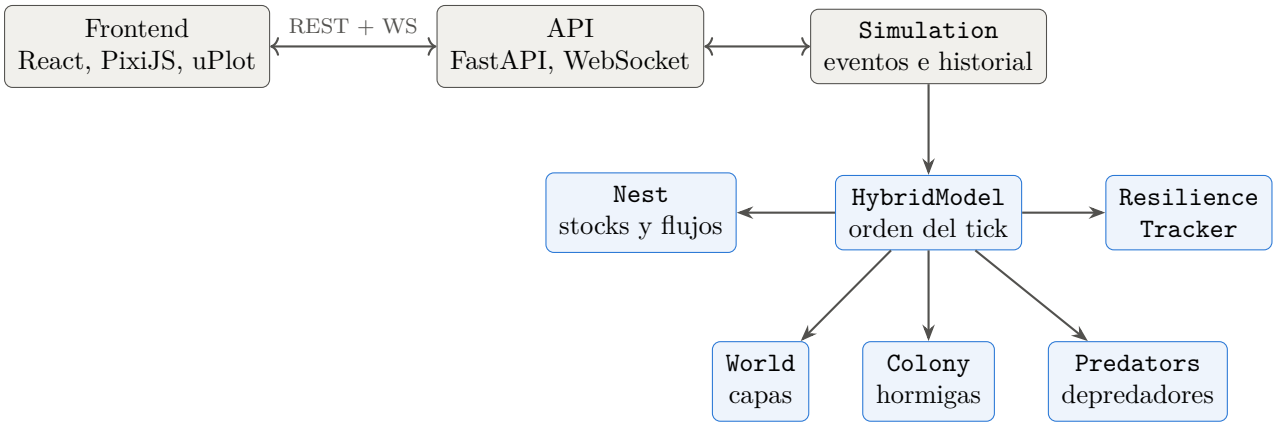


Figura 1: Arquitectura del sistema. Los componentes azules forman el modelo híbrido; `HybridModel` los ejecuta en un orden fijo en cada tick.

4.2. El mundo

El mundo es una cuadrícula de 200×200 celdas. Cada capa es un arreglo `float32` de forma $(\text{alto}, \text{ancho})$: comida, obstáculos, agua, humedad y tres feromonas (hacia la comida, hacia el nido y de alarma). Las reglas del mundo son:

- **Comida.** Hay n fuentes circulares con una capacidad por celda. Se regeneran a una tasa proporcional a la humedad media de la fuente y decaen un 0,1 % por tick multiplicado por la sequedad.
- **Feromonas.** Las hormigas depositan en su celda; en cada tick la feromona se difunde a las celdas vecinas y se evapora. Cada tipo tiene su propia tasa de evaporación.
- **Humedad.** Se acerca a un objetivo con un 2 % por tick. Una sequía baja el objetivo a $1 -$ intensidad en su región.
- **Agua.** Una inundación fija un objetivo de agua en su región; el agua se propaga hacia celdas vecinas y no atraviesa obstáculos.
- **Obstáculos.** Bloquean el paso y eliminan la comida y la feromona de las celdas que ocupan. El usuario puede dibujarlos o borrarlos en ejecución, salvo en el nido.

4.3. Las hormigas (ABM)

Cada hormiga es una fila en arreglos paralelos de NumPy (posición, rumbo, rol, estado, carga, energía, edad). Esto permite actualizar a toda la colonia con operaciones vectorizadas, sin bucles sobre agentes.

Roles. Hay tres: *forager* (recolectora), *scout* (exploradora, que ignora el rastro el 80 % del tiempo y gira más) y *soldier* (soldado, que no huye y daña a los depredadores cercanos). Al nacer, una hormiga es soldado con probabilidad `soldier_fraction`, y exploradora con un 10 % del resto.

Estados. Cuatro estados:

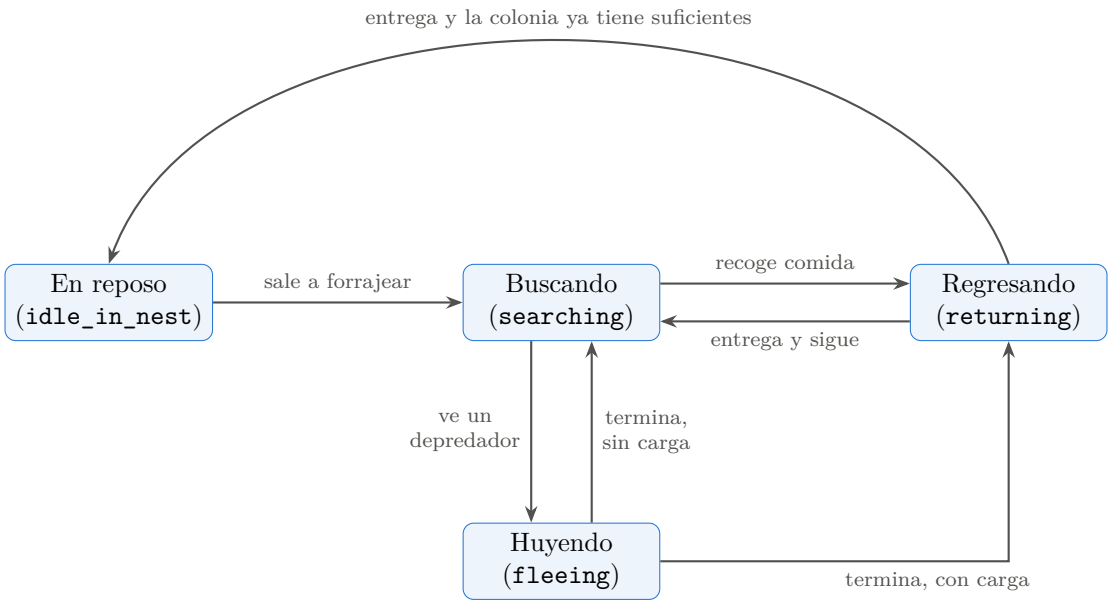


Figura 2: Máquina de estados de una hormiga. Una hormiga que regresa sin carga (por poca energía) pasa a reposo al llegar al nido.

Reglas de forrajeo.

- *Búsqueda.* La hormiga tiene tres sensores (izquierda, centro, derecha) a 3 celdas y ±45°. Si la feromona de comida menos la de alarma supera 0,05 en algún sensor, gira hacia el mayor; si no, gira al azar hasta 15° por tick. Si se aleja más del 35 % del lado del mundo (el doble de lejos las exploradoras) sin rastro, regresa hacia el nido.
- *Recogida.* Recoge solo si la celda tiene al menos el 30 % de su capacidad de carga. Esto evita que las hormigas regresen con migajas.
- *Regreso.* La hormiga con carga se orienta hacia el nido y deposita feromona de comida.
- *Entrega.* Al llegar al nido entrega la carga; el modelo SD la recibe como ingreso de comida.
- *Energía.* Fuera del nido pierde `energy_decay`·(2 − humedad) por tick; dentro del nido recupera 2 por tick si el nido tiene comida. Si la energía baja del 25 % mientras busca, regresa.
- *Muertes.* Por vejez (esperanza de vida con variación individual), por agotamiento de energía ("hambre"), por ahogamiento en agua (probabilidad proporcional al nivel de agua) o por un depredador.

4.4. Los depredadores

Un depredador puede estar *wandering*, *hunting* o *eating*. Si hay una hormiga activa (fuera del nido) dentro de su radio de visión, la persigue; la captura si llega a menos de máx$(1, v)$ celdas, donde v es su velocidad, con probabilidad `attack_probability`, y entonces come durante 20 ticks. Gasta energía en cada tick; los soldados a menos de 1,5 celdas le restan energía, y muere al agotarla. No entra al nido.

4.5. El nido (SD)

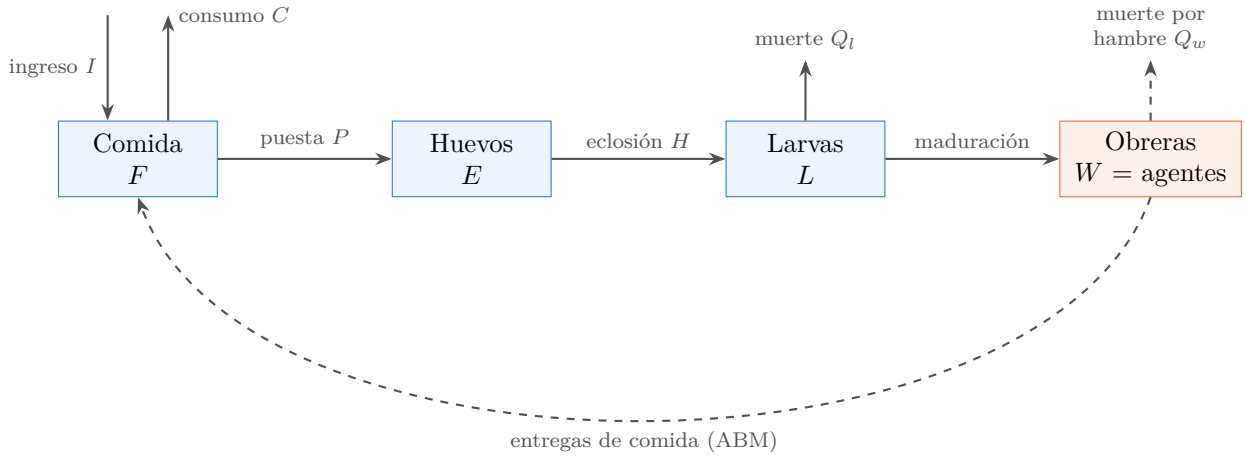


Figura 3: Diagrama de stocks y flujos del nido. Las obreras (naranja) son los agentes vivos; las flechas punteadas conectan el nido con el ABM.

Sea F la comida, E los huevos, L las larvas y W las obreras vivas, y dt el paso de integración. Los flujos son:

$$I = \frac{\text{comida entregada}}{dt} \qquad D = W\,c_w + L\,c_l \tag{1}$$

$$C = \min\!\left(D,\ \frac{F}{dt} + I\right) \qquad h = \frac{D - C}{D} \quad (h = 0 \text{ si } D = 0) \tag{2}$$

$$P = p_{\max}\,\frac{F}{F + K_F} \qquad H = \frac{E}{\tau_e} \tag{3}$$

$$Mat = \frac{L}{\tau_l} \qquad Q_l = \mu\,h\,L, \qquad Q_w = \mu\,h\,W \tag{4}$$

donde c_w y c_l son los consumos por obrera y por larva, $p_{\max}$ la puesta máxima, K_F la comida a media saturación, τ_e y τ_l las duraciones de huevo y larva, μ la mortalidad por hambre y $h \in [0, 1]$ el hambre. La actualización de Euler es

$$F \leftarrow F + (I - C)\,dt, \qquad E \leftarrow E + (P - H)\,dt, \qquad L \leftarrow L + (H - Q_l)\,dt - n_{\text{nuevas}}, \tag{5}$$

y todos los stocks se limitan a cero o más.

4.6. Acoplamiento entre SD y ABM

El acoplamiento es el centro del modelo híbrido y respeta dos invariantes:

1. W **es igual al número de agentes vivos** al final de cada tick. Las obreras no se integran.
2. **Cada larva que madura se convierte en exactamente una hormiga real.**

Para lograrlo con flujos continuos y agentes discretos, el nido guarda dos acumuladores fraccionales:

$$\text{maduración}_{k+1} = \text{maduración}_k + Mat \cdot dt, \quad n_{\text{nuevas}} = \lfloor \text{maduración} \rfloor \tag{6}$$

$$\text{hambre}_{k+1} = \text{hambre}_k + Q_w \cdot dt, \quad n_{\text{muertes}} = \lfloor \text{hambre} \rfloor \tag{7}$$

n_{nuevas} se limita por el cupo `max_ants` $-W$ y por las larvas disponibles; si la colonia está llena, las larvas esperan. Si el ABM crea menos hormigas de las pedidas, la diferencia vuelve al stock de larvas. Solo se restan los enteros efectivamente usados, de modo que la suma de hormigas nacidas coincide con la maduración integrada dentro de una unidad (se verifica en las pruebas).

Fracción de forrajeo. La estrategia de la colonia depende de su tamaño y su hambre:

$$f = f_{\min} + (f_{\max} - f_{\min})\,\frac{W}{W + K_W}, \qquad f_{\text{final}} = \min\!\big(f_{\max},\ f + 0{,}3\,h\big) \tag{8}$$

donde K_W es el tamaño de colonia con fracción intermedia y h es el hambre del tick anterior. Las hormigas en reposo salen a forrajear hasta que la fracción activa alcanza f_{final}.

Orden de un tick. `HybridModel` ejecuta, en este orden:

1. `world.update`: regeneración de comida y dinámica de agua y humedad.
2. `colony.step`: movimiento, sensado, recogida, entrega, depósitos y muertes.
3. `predators.step`: caza, combate con soldados y muerte de depredadores.
4. `world.update_pheromones`: difusión y evaporación.
5. `nest.step`: integración SD del nido.
6. `colony.starve` y `colony.spawn` según el reporte del nido.

4.7. Perturbaciones y métrica de resiliencia

Las perturbaciones se programan por tick de inicio, duración, intensidad y región:

- **Sequía:** baja el objetivo de humedad de la región a 1 − intensidad.
- **Inundación:** fija un objetivo de agua; las hormigas en celdas con agua mayor a 0,3 pueden ahogarse.
- **Oleada de depredadores:** crea depredadores en celdas libres de la región, sin superar `max_predators`, y los retira al terminar.

La resiliencia se mide con un `ResilienceTracker` que actualiza en cada tick, y sin depender del historial muestreado, el pico de población, el mínimo y su tick, la eficiencia media de forrajeo y, por evento, la población previa, el mínimo durante el evento y los ticks de recuperación (primer tick posterior al fin del evento con población mayor o igual a la previa). Esto evita la pérdida de precisión cuando el historial se compacta.

4.8. Interfaz y contrato

La API expone rutas REST para crear y controlar simulaciones, editar el mundo, programar perturbaciones y consultar historial y resumen, y un WebSocket que transmite cuadros con hormigas, depredadores, stocks, flujos y métricas, además de las capas de la cuadrícula codificadas en bytes. El contrato se exporta a `contract/openapi.json` y el frontend genera sus tipos a partir de él, de modo que un cambio incompatible se detecta al compilar.

El frontend ofrece: el mundo dibujado con PixiJS (feromonas como mapa de calor, hormigas con posición interpolada entre cuadros), una barra superior con controles y contadores, una paleta para dibujar obstáculos, colocar comida y depredadores y borrar, un diagrama animado de stocks y flujos del nido, gráficas en vivo, un panel de parámetros editables, una línea de tiempo de perturbaciones, una tarjeta de resiliencia y atajos de teclado.

4.9. Determinismo y rendimiento

Hay un único generador `numpy.random.Generator` creado a partir de la semilla; ningún componente usa el generador global. Misma semilla y mismos comandos producen la misma corrida, lo que permite comparar escenarios y reproducir resultados. Un tick completo con 2000 hormigas, 50 depredadores y un mundo de 200×200 tarda 7.2 ms en promedio sobre 200 ticks (el requisito era menos de 15 ms).

4.10. Parámetros de los escenarios

Los valores por defecto del esquema no sostienen una colonia (sección 6), por lo que los cuatro escenarios usan una configuración común calibrada:

Tabla 3: Parámetros de la configuración calibrada. Lo no listado conserva su valor por defecto.

Grupo	Parámetro	Valor
Mundo	Fuentes de comida / radio / regeneración	12 / 8 / 3,0
Feromonas	Evaporación (comida y hogar)	0,005
Hormigas	Velocidad / gasto de energía	2,0 / 0,45
Hormigas (defecto)	Iniciales / máximo / esperanza de vida	100 / 2000 / 5000
Nido	Comida inicial	1000
Nido	Consumo por obrera / por larva	0,005 / 0,01
Nido	Puesta máxima $p_{\text{máx}}$	0,05
Nido	Mortalidad por hambre μ	0,002
Nido (defecto)	K_F / τ_e / τ_l	100 / 200 / 400
Sequía	Intensidad / inicio / duración	0,9 / 6000 / 3000
Inundación	Intensidad / inicio / duración / región	0,5 / 5000 / 150 / 80×80
Invasión	Depredadores iniciales / velocidad / ataque	2 / 2,0 / 0,6
Invasión	Oleada: cantidad / inicio / duración	30 / 3000 / 5000

5. Herramientas aplicadas

5.1. Herramientas vistas en clase

Del curso usamos el modelado basado en agentes, la dinámica de sistemas (stocks, flujos, retrasos y retroalimentación, integración de Euler), la simulación de eventos discretos con avance de tiempo fijo, la generación de números pseudoaleatorios con semilla y la validación por réplicas con distintas semillas.

5.2. Herramientas que no se vieron en clase

Tabla 4: Tecnologías utilizadas y su papel.

Herramienta	Papel en el proyecto
Python 3.13, NumPy	Motor de simulación con operaciones vectorizadas sobre arreglos
FastAPI, Uvicorn, WebSocket	API REST y transmisión de cuadros en tiempo real
Pydantic, OpenAPI	Esquemas validados y contrato exportable
React 19, TypeScript, Vite	Aplicación web, tipado estático y empaquetado
PixiJS, uPlot	Dibujo del mundo en GPU y gráficas de series de tiempo
Zustand, TanStack Query	Estado del cliente y consultas con caché
`openapi-typescript`	Tipos del frontend generados desde el contrato
pytest, Vitest	136 pruebas del backend y 66 del frontend
Matplotlib, LaTeX	Figuras y este documento
Git y GitHub	Control de versiones con commits convencionales
Asistente de IA generativa	Implementación, pruebas, diagnóstico y redacción (sección 5.3)

Vectorización con NumPy. Simular 2000 hormigas con bucles de Python en cada tick es demasiado lento para una animación fluida. NumPy permite guardar cada atributo de la colonia en un arreglo y aplicar la misma operación a todos los agentes a la vez (por ejemplo, sumar el ruido de giro a todos los rumbos o muestrear las tres feromonas en los tres sensores de todas las hormigas). Los bucles de Python solo se usan sobre depredadores, eventos y fuentes de comida, que son pocos. Las hormigas que eliminan se quitan con máscaras booleanas, y los identificadores no se reutilizan, porque el frontend interpola posiciones por identificador.

FastAPI y Pydantic. FastAPI construye la API a partir de funciones tipadas y genera la especificación OpenAPI automáticamente. Pydantic valida los parámetros, aplica sus límites y marca cuáles pueden editarse en vivo. El mismo esquema alimenta la documentación, la validación y los tipos del frontend, de modo que un cambio de contrato rompe la compilación en lugar de fallar en ejecución.

WebSocket. Una conexión persistente permite al servidor enviar cada cuadro sin que el cliente pregunte. Cada mensaje lleva un número de secuencia consecutivo; si un cliente lento llena su cola, el servidor descarta cuadros y capas (no los avisos), y un salto en la secuencia indica que el cliente debe pedir el estado completo.

PixiJS, React y Zustand. PixiJS dibuja miles de sprites con aceleración por GPU y es la base del mundo animado; React organiza los paneles; Zustand mantiene el estado de la simulación fuera de los componentes para que un cuadro nuevo no repinte toda la interfaz. El cliente interpola las posiciones entre cuadros para que el movimiento se vea continuo aunque el servidor envíe pocos cuadros por segundo.

pytest y Vitest. Las pruebas del backend verifican propiedades del modelo, no solo que el código corra: la solución de Euler a mano durante 10 ticks, la suma de hormigas nacidas contra la maduración integrada, que W sea igual a las hormigas vivas durante 2000 ticks, que dos corridas con la misma semilla produzcan cuadros idénticos y que el mínimo del resumen sea exacto aunque el historial se compacte.

5.3. Uso de IA generativa

Usamos un asistente de IA (Claude Code) como programador y revisor, no como oráculo. El trabajo se estructuró así: el equipo redactó una guía por paquete con las ecuaciones, las interfaces y los criterios de aceptación; el asistente implementó contra esa guía; las pruebas y las mediciones decidieron si el resultado era correcto. Una regla fija del equipo fue que el asistente nunca hace *commit*: prepara los cambios y propone el mensaje, y una persona del equipo revisa y confirma.

[Completar: prompts de la propuesta, del contrato de la API, del frontend y de las guías 1 (mundo) y 2 (agentes), con la misma estructura de la tabla siguiente. Los de la guía 3 y de este documento están redactados abajo a partir de esta sesión.]

Tabla 5: Prompts relevantes de la guía 3, resultado y evaluación crítica.

Prompt (resumen)	**Qué ocurrió**	**Evaluación**
"Sigue la guía 3 (`modelo-3-nido.md`); dime si requieres información."	El asistente leyó la guía, revisó el repositorio y vio que las guías 1 y 2 ya estaban integradas, por lo que omitió la fachada provisional. Implementó `Nest`, `HybridModel` y `ResilienceTracker` con sus pruebas pasando.	**Satisfactorio.** La guía traía ecuaciones, firmas e invariantes, y el resultado se comprobó contra una solución de Euler escrita a mano. Aun así, el asistente tomó una decisión de alcance (saltar la fachada) que nosotros tuvimos que aprobar.
"Opción 1: retoca el forrajeo de `Colony`." (tras informar que los cuatro escenarios colapsaban)	Al calibrar, la colonia base se extinguía hacia el tick 700 en las cinco semillas. El asistente midió en lugar de suponer: probó configuraciones (fuentes, regeneración, consumo, feromonas) sin éxito, descubrió que las hormigas con carga se alejaban del nido, que perdían la comida al huir y que recogían migajas, y corrigió cada causa con una prueba.	**Mixto, y por eso instructivo.** Varias hipótesis iniciales resultaron falsas: orientar las hormigas al nido no bastó, y subir la carga por viaje no mejoró la entrega. Cada hipótesis se descartó con una medición, no con intuición.
(misma tarea; prueba nueva)	Al escribir la prueba "las hormigas que huyen y llevan comida vuelven a regresar", falló: sin depredadores en el mundo, las hormigas quedaban en estado de huida para siempre.	**Error preexistente, hallado por una prueba.** Es un ejemplo de por qué las pruebas escritas por la IA deben comprobar comportamiento.
"Nunca hagas commits por tu cuenta; solo `git add` y dime el mensaje en inglés."	El asistente dejó de confirmar, separó el trabajo en commits temáticos y entregó cada mensaje.	**Satisfactorio.** Una instrucción de proceso se siguió de forma consistente durante toda la sesión.
"Empieza con el documento en LaTeX."	El asistente generó este documento y los resultados con un script reproducible (`backend/scripts/results.py`), de modo que las tablas y las figuras provienen de corridas reales.	**Satisfactorio con una salvedad.** Los datos son reales, pero la redacción de antecedentes y la bibliografía necesitan revisión humana.

Qué funcionó y qué no, y por qué. La IA funcionó bien cuando la tarea estaba especificada de forma verificable: ecuaciones explícitas, firmas de métodos, invariantes y criterios numéricos. Falló, o tardó, cuando la tarea era *emergente*: que una colonia sobreviva depende de la interacción de decenas de reglas, y ninguna lectura del código lo garantiza. Allí fue necesario medir (entregas por 100 ticks, distancia al nido de las hormigas con carga, carga media por recogida) y descartar hipótesis. También observamos que el asistente podía dar por buena una explicación plausible; por eso cada corrección se validó con una prueba o con una repetición de la calibración completa.

Cómo evaluamos las respuestas. Con cuatro filtros: (1) pruebas automáticas que comprueban propiedades y no solo ejecución; (2) comparación con una referencia independiente (la solución de Euler a mano y los equilibrios analíticos de la sección 6); (3) réplicas con cinco semillas para no confiar en una sola corrida; y (4) revisión humana de cada *commit* antes de confirmarlo.

Cómo contribuyó la IA. Aceleró la escritura de código vectorizado, de pruebas y de herramientas de diagnóstico, y permitió explorar muchas configuraciones en poco tiempo. La responsabilidad de las decisiones de modelado (qué es un agente, qué es un stock, qué criterios de calibración aceptar) y la comprensión del modelo siguieron siendo del equipo; la sección 6 muestra, con los equilibrios analíticos, que entendemos por qué el modelo se comporta como lo hace.

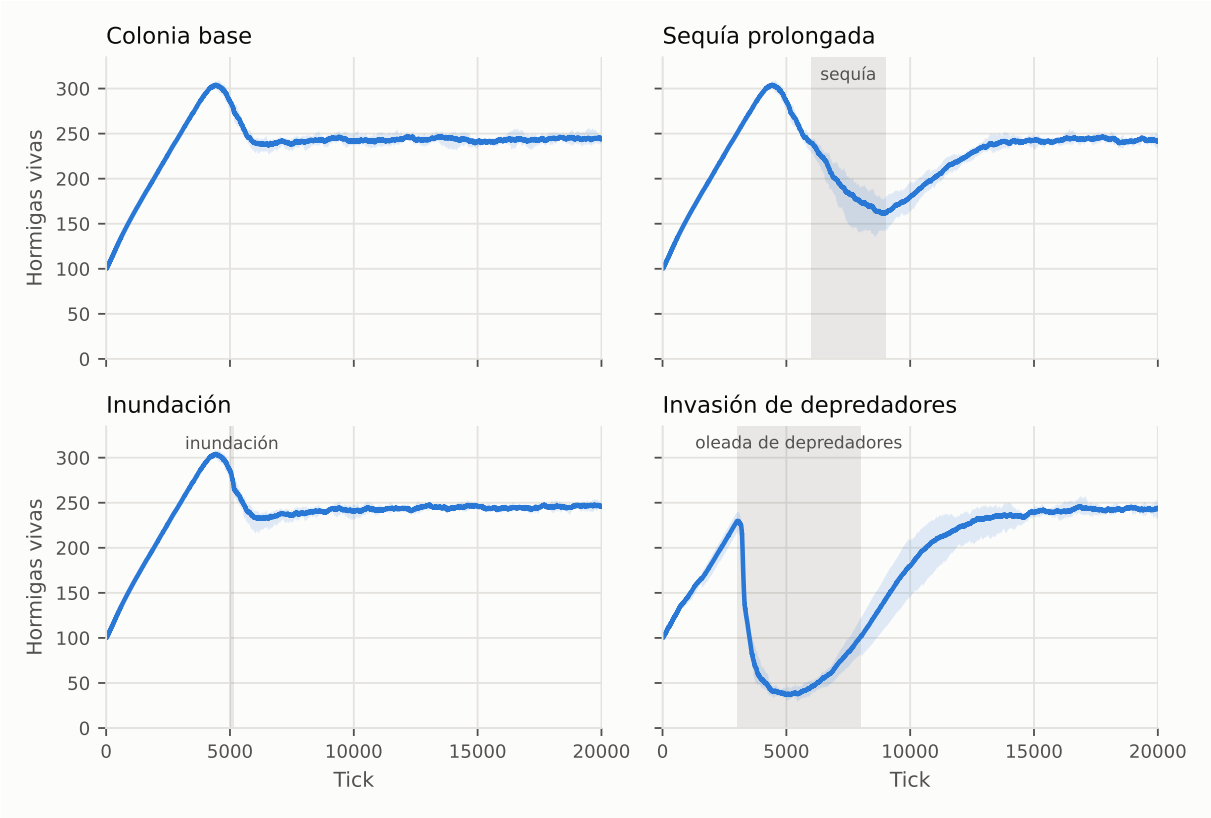


Figura 4: Población de hormigas por escenario. La línea es la media de cinco semillas y la banda va del mínimo al máximo; la zona gris marca el evento.

6. Resultados

6.1. Método

Cada escenario se simuló con las semillas 1 a 5 y 20 000 ticks (el máximo por defecto). Reportamos medias sobre las cinco semillas. Los resultados salen de `backend/scripts/results.py` y se guardan en `docs/informe/` (`resultados.json`, `ablacion.json`, `tablas.tex` y las figuras), por lo que cualquier número de este documento puede reproducirse.

6.2. Supervivencia por escenario

Tabla 6: Resumen por escenario (media de cinco semillas). "Caída" es la reducción máxima respecto de la población al inicio del evento; "Recuperada" cuenta semillas donde la población volvió a la previa.

Escenario	Viva	Final	Pico	Caída	Recup.	Comida entregada
Colonia base	5/5	244	306	–	–	45 472
Sequía prolongada	5/5	242	306	34 %	5/5	39 076
Inundación	5/5	246	306	6 %	0/5	43 889
Invasión de depredadores	5/5	244	252	85 %	5/5	31 061

Colonia base. La población crece desde 100 hasta un pico de 306 cerca del tick 4500 y se estabiliza en 244 ± 4 (entre 238 y 248 según la semilla), entre 2,4 y 2,5 veces la inicial. La banda entre semillas es muy estrecha: el sistema es robusto frente al azar.

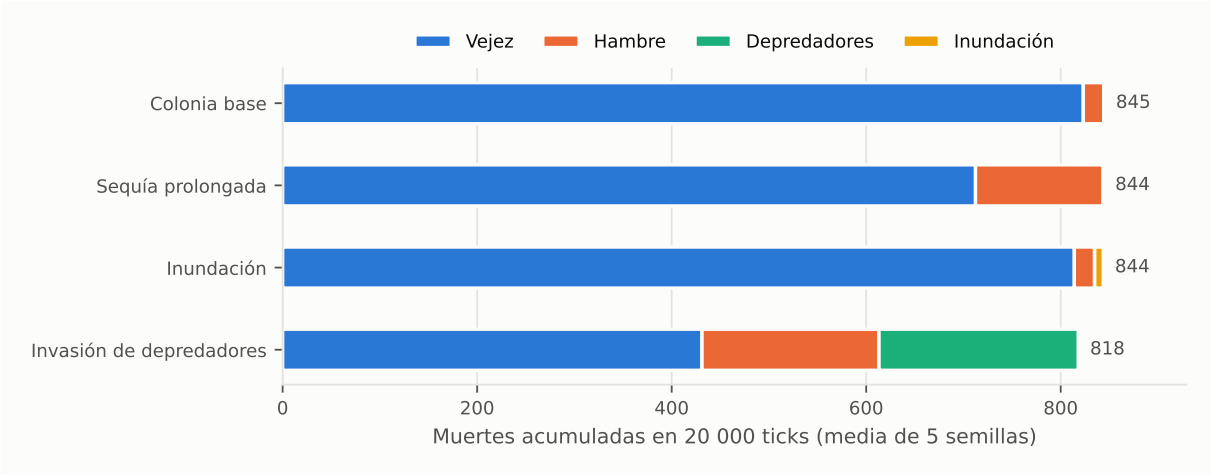


Figura 5: Composición de las muertes por escenario. La longitud total es parecida en los cuatro: la colonia reemplaza lo que pierde, pero cambia de dónde vienen las pérdidas.

Sequía. La población cae en promedio un 34 % respecto de su valor al inicio de la sequía (de 25 a 45 % según la semilla) y se recupera en las cinco semillas, entre 3900 y 5800 ticks después de terminar. La sequía actúa por dos caminos: baja la regeneración de la comida y encarece la energía de las hormigas fuera del nido, así que mueren más por hambre (131 por corrida, frente a 21 en la colonia base).

Inundación. Mueren entre 6 y 17 hormigas por ahogamiento y la colonia sobrevive en las cinco semillas. En la tabla 6 la recuperación aparece como 0/5, pero es un efecto de la métrica: la inundación empieza en el tick 5000, cuando la colonia todavía está en la bajada del pico (cerca de 285 hormigas), por encima del equilibrio de largo plazo (cerca de 245). La población vuelve a su equilibrio, pero no supera la cifra previa. Es una limitación de definir la recuperación como "volver a la población previa".

Invasión de depredadores. La oleada de 30 depredadores reduce la población, en promedio, un 85 % (hasta 29–43 hormigas) y la colonia se recupera en las cinco semillas. Los depredadores causan el 25 % de las muertes (22 a 27 % por semilla); el resto es hambre (el miedo y la huida interrumpen el forrajeo) y vejez.

6.3. Causas de muerte

Tabla 7: Muertes acumuladas por causa en 20 000 ticks (media de cinco semillas).

Escenario	Vejez	Hambre	Depredadores	Inundación
Colonia base	823	21	0	0
Sequía prolongada	712	131	0	0
Inundación	814	21	0	9
Invasión de depredadores	431	182	205	0

6.4. Verificación contra equilibrios analíticos

El modelo SD tiene equilibrios que se pueden calcular a mano, y sirven para comprobar la implementación. Con comida abundante, la puesta es $P \approx p_{\text{máx}} = 0{,}05$ huevos por tick. Entonces:

- Huevos en equilibrio: $E^* = P\,\tau_e = 0{,}05 \times 200 = 10$.
- Larvas en equilibrio: $L^* = P\,\tau_l = 0{,}05 \times 400 = 20$.

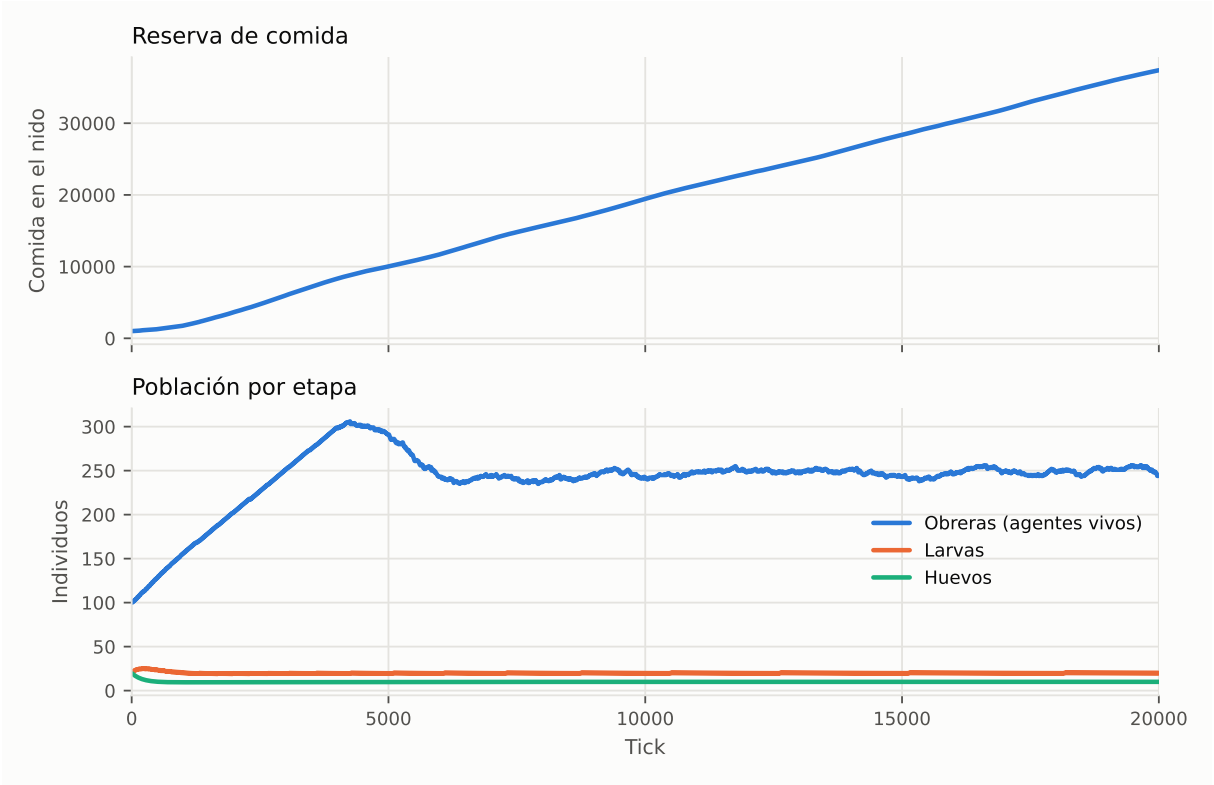


Figura 6: Stocks del nido en la colonia base (semilla 1). Arriba, la reserva de comida; abajo, obreras, larvas y huevos. Huevos y larvas convergen a los equilibrios analíticos 10 y 20.

- Obreras: el flujo de nacimientos es P y la vida media es de 5000 ticks, así que $W^* \approx 0{,}05 \times 5000 = 250$.

La figura 6 muestra que los huevos y las larvas convergen a 10 y 20, y la población media final es de 244, a un 2 % del valor teórico. El pico de 306 cerca del tick 4500 no contradice el equilibrio y es consistente con la muerte de la cohorte inicial: las 100 hormigas iniciales se crean con edad cero y tienen una esperanza de vida cercana a 5000 ticks. No verificamos esta explicación con un experimento aparte.

6.5. El proceso iterativo: qué cambió con las correcciones

La tabla 8 compara tres versiones con las mismas semillas. La primera usa la misma configuración calibrada pero el comportamiento de `Colony` anterior a las correcciones del forrajeo. La segunda usa el código final con los parámetros por defecto del esquema. La tercera es la versión final.

Tabla 8: Colonia base con cinco semillas y 20 000 ticks, según la versión del código y la configuración.

Versión	Vivas	Colapso (ticks)	Pico medio	Comida entregada
Forrajeo anterior, configuración calibrada	0/5	2877–11 452	130	1131
Código final, parámetros por defecto	0/5	753–840	108	22
Código final, configuración calibrada	5/5	–	306	45 472

Dos conclusiones. Primero, las correcciones del forrajeo son necesarias: sin ellas, la misma configuración entrega cuarenta veces menos comida y la colonia se extingue. Segundo, los parámetros por defecto del esquema no son viables (regeneración de comida de cero y consumo alto); esto es una limitación conocida

y la tratamos en la sección 7.

Los defectos del forrajeo que se corrigieron, y cómo se detectó cada uno, fueron: las hormigas con carga seguían el máximo local de feromona de hogar y se alejaban del nido; las buscadoras se alejaban sin límite en caminatas casi rectas; al huir con carga, las hormigas la perdían al recoger de nuevo; recogían migajas de 0,02 unidades; y, cuando ya no quedaban depredadores, las hormigas en huida no volvían nunca a su estado normal. Además, con velocidades de 2 celdas por tick, el radio de captura fijo de 1 celda hacía que los depredadores sobrepasaran a su presa y casi no cazaran, por lo que el radio ahora escala con la velocidad.

6.6. Calidad técnica

Tabla 9: Verificaciones técnicas.

Verificación	**Resultado**
Pruebas del backend (pytest)	136 pasan
Pruebas del frontend (Vitest)	66 pasan
Compilación del frontend y tipos desde el contrato	Sin errores ni cambios
Determinismo (misma semilla, 500 ticks, con obstáculo, comida y sequía)	Cuadros idénticos
Invariante W = hormigas vivas, 2000 ticks	Se cumple en cada tick
Mínimo del resumen exacto con el historial compactado	Se cumple
Tiempo de un tick (2000 hormigas, 50 depredadores, 200×200)	7.2 ms (límite 15 ms)

6.7. Criterios de calibración

Tabla 10: Criterios acordados para los escenarios (semillas 1 a 5, 20 000 ticks) y resultado.

Escenario	**Criterio**	**Observado**	**Cumple**
Base	Sobrevive; final entre 1,2 y 5 veces la inicial	5/5 vivas; 2,4 a 2,5 veces	Sí
Sequía	Caída $\geq 15\,\%$; recuperación en ≥ 3 de 5	Caída de 25 a 45 %; 5/5	Sí
Inundación	Muertes por inundación > 0; sobrevive	6 a 17 muertes; 5/5 vivas	Sí
Invasión	Depredadores $> 20\,\%$ de las muertes; sobrevive en ≥ 3 de 5	25 %; 5/5 vivas	Sí

7. Conclusión

Construimos un modelo híbrido en el que los agentes y el nido se retroalimentan: lo que entregan las hormigas alimenta la puesta, la maduración repone a los agentes y el tamaño y el hambre de la colonia cambian su estrategia de forrajeo. El acoplamiento respeta sus invariantes en cada tick y sus equilibrios coinciden con el cálculo analítico (huevos 10, larvas 20 y 244 obreras frente a 250). Con cinco semillas, la colonia sobrevive a una sequía, una inundación y una invasión, y cada perturbación deja una huella distinta en las causas de muerte, como el planteamiento original esperaba.

Hallazgos principales:

1. El equilibrio de población de la colonia calibrada lo fija la demografía (puesta $\times$ esperanza de vida), no la comida: la reserva del nido crece sin límite (figura 6).

2. En nuestros experimentos, la sequía afecta a la colonia sobre todo porque encarece la energía de las hormigas fuera del nido; la baja regeneración de comida pesa menos porque la reserva del nido amortigua.
3. La invasión de depredadores mata menos por captura directa que por interrupción del forrajeo: el hambre es casi tan importante como los depredadores.
4. La eficacia del forrajeo dependía de detalles de movimiento fáciles de pasar por alto. Sin las correcciones, la misma configuración entrega cuarenta veces menos comida.

7.1. Limitaciones

- **Parámetros por defecto no viables.** Con la regeneración de comida en cero, la colonia colapsa en unos 800 ticks. Los escenarios usan una configuración calibrada. No cambiamos los valores por defecto porque forman parte del contrato de la API.
- **Reserva del nido sin tope.** La comida almacenada crece sin límite, lo que amortigua las sequías. Un tope de almacenamiento o una regulación del forrajeo según la reserva sería más realista.
- **Régimen limitado por comida inestable.** Cuando subimos el consumo hasta que la comida sí limitaba a la colonia, el sistema oscilaba y varias semillas se extinguían. El modelo estable es el que se limita por demografía.
- **Dependencia de la semilla.** Las fuentes de comida se colocan al azar; en semillas con fuentes lejanas el forrajeo es difícil. Añadimos fuentes para dar robustez, pero la sensibilidad persiste.
- **Calibración.** Los parámetros se ajustaron con las semillas 1 a 5 y no se validaron con semillas nuevas ni con datos empíricos.
- **Métrica de recuperación.** Definirla como "volver a la población previa" depende del momento del evento (caso de la inundación).
- **Verificación manual de la interfaz** pendiente de documentar: la revisión visual del frontend y las capturas para el video.

7.2. Trabajo futuro

Colonias rivales (como en los modelos de referencia con energía), feromonas repelentes, tope de almacenamiento, comparación con datos de campo, un análisis de sensibilidad de parámetros y validación con semillas no usadas en la calibración.

Referencias

[1] Ant Adaptation. NetLogo Models Library. `https://ccl.northwestern.edu/netlogo/models/AntAdaptation`. Añade energía y colonias rivales. COMPLETAR autores y año. Consultado el 1 de noviembre de 2026.

[2] Ants2015. NetLogo Community Models. `https://ccl.northwestern.edu/netlogo/models/community/Ants2015`. La energía del nido produce nuevas hormigas (puente ABM–SD). COMPLETAR autores y año. Consultado el 1 de noviembre de 2026.

[3] Completar título: forrajeo de hormigas cosechadoras *Pogonomyrmex* según la distribución de comida y el tamaño de la colonia. University of New Mexico Digital Repository. `https://digitalrepository.unm.edu/cs_etds/57`. COMPLETAR autores y año. Consultado el 1 de noviembre de 2026.

[4] Completar título: resiliencia de la colonia ante depredadores y evaporación de feromonas. Vilnius University ePublications. `https://epublications.vu.lt/object/elaba:238880877/index.html`. Tesis. COMPLETAR autor y año. Consultado el 1 de noviembre de 2026.

[5] Completar título: reclutamiento según el tamaño de la colonia (abm que reformula un modelo diferencial previo). Computational Science, University of Amsterdam. `https://students.computationalscience.nl/wp-content/uploads/2019/09/Ants.pdf`, 2019. COMPLETAR autores. Consultado el 1 de noviembre de 2026.

[6] M. Holcombe et al. Completar título: feromonas atractivas y repelentes y robustez de la colonia. CoMSES Net. https://catalog.comses.net/publications/1703. COMPLETAR año y lista de autores. Consultado el 1 de noviembre de 2026.

[7] Uri Wilensky. NetLogo Ants model. Center for Connected Learning and Computer-Based Modeling, Northwestern University. https://ccl.northwestern.edu/netlogo/models/Ants, 1997. Modelo clásico de forrajeo con feromonas. Consultado el 1 de noviembre de 2026.